

NUMBER SYSTEMS

Unit 2

BASE-N REPRESENTATIONS

Base 2 (Binary) Representation

$$B = b_{n-1}b_{n-2} \cdots b_1b_0$$

$$\begin{aligned} V(B) &= b_{n-1} \times 2^{n-1} + b_{n-2} \times 2^{n-2} + \cdots + b_1 \times 2^1 + b_0 \times 2^0 \\ &= \sum_{i=0}^{n-1} b_i \times 2^i \end{aligned}$$

Base 8 (Octal) Representation

Numbers in range 0,1,2,...7

Example: $(317)_8 = 8^2 \times 3 + 8^1 \times 1 + 8^0 \times 7$

Base 16 (Hex) Representation

Numbers in range 0,1..9,A,B..F

Example: $(A3)_{16} = 16^1 \times 10 + 16^0 \times 3$

Table 3.1 Numbers in different systems.

Decimal	Binary	Octal	Hexadecimal
00	00000	00	00
01	00001	01	01
02	00010	02	02
03	00011	03	03
04	00100	04	04
05	00101	05	05
06	00110	06	06
07	00111	07	07
08	01000	10	08
09	01001	11	09
10	01010	12	0A
11	01011	13	0B
12	01100	14	0C
13	01101	15	0D
14	01110	16	0E
15	01111	17	0F
16	10000	20	10
17	10001	21	11
18	10010	22	12

HEX USED AS SHORTHAND FOR BINARY

Computers used binary representations. For examples, “ints” in C are 32-bit binary numbers.

But writing down 32-bits is tedious, so we often use a hexadecimal representation instead.

Q: How many hexadecimal numbers needed to represent a 32-bit binary value? *Hint: how many binary numbers do you need for a single hex number?*

EXAMPLE

Convert the following binary value to hexadecimal.

$$(1010111100100101)_2 = (AF25)_{16}$$

Strategy: group into sets of 4 bits each. Convert each set of 4 bits into hex equivalent.

$\underbrace{1010}_{A}$

$\underbrace{1111}_{F}$

$\underbrace{0010}_2$

$\underbrace{0101}_5$

ADDING BINARY VALUES

Binary additions are very similar to the way we perform decimal additions!

$$\begin{array}{rcccccc} & \boxed{+1} & \boxed{+1} & & \boxed{+1} & & \\ & 1 & 1 & 0 & 1 & = & 13 \\ + & 0 & 1 & 0 & 1 & = + & 5 \\ \hline 1 & 0 & 0 & 1 & 0 & = & 18 \end{array}$$

ADDING BINARY VALUES

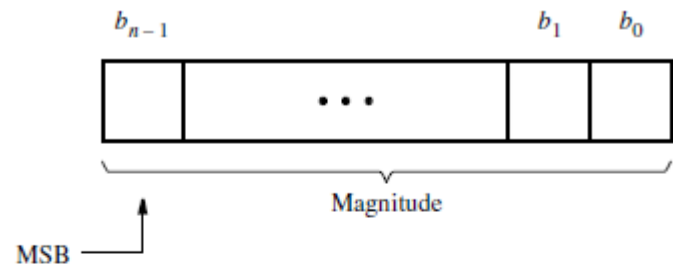
Binary additions are very similar to the way we perform decimal additions!

$$\begin{array}{rcccccc} & \boxed{+1} & \boxed{+1} & & \boxed{+1} & & \\ & 1 & 1 & 0 & 1 & = & 13 \\ + & 1 & 1 & 0 & 1 & = & +13 \\ \hline 1 & 1 & 0 & 1 & 0 & = & 26 \end{array}$$

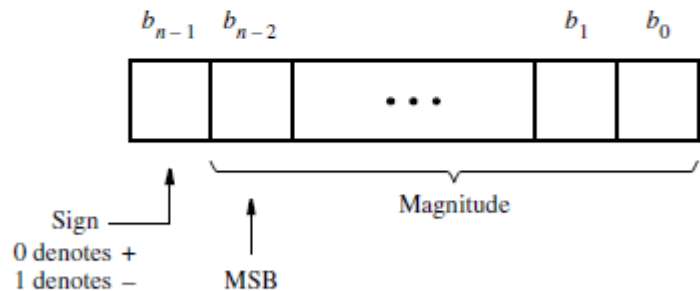
SIGNED ARITHMETIC

What about negative numbers?

- **Sign magnitude representation**
- **The most-significant bit (MSB) is zero for positive numbers and 1 for negative numbers**
- **The remaining bits indicate the magnitude**



(a) Unsigned number



(b) Signed number

Figure 3.7 Formats for representation of integers.

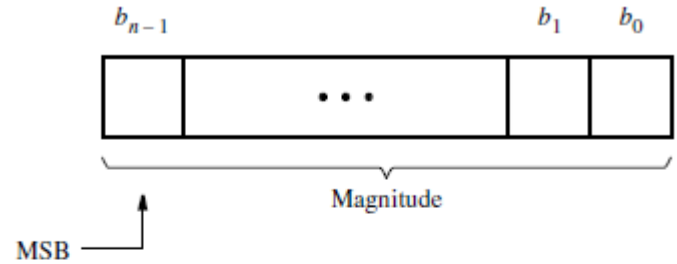
EXAMPLE

- Write down the numbers 2 and -7 using 6-bit SMR representation.

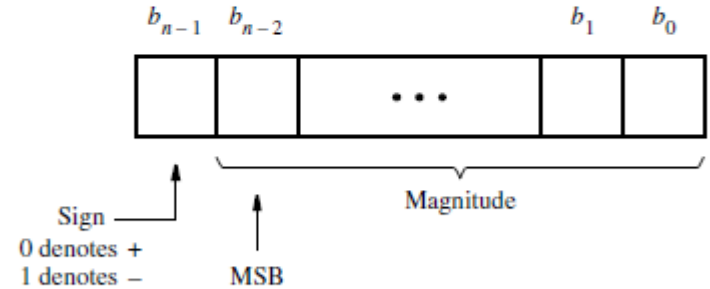
$$2 = 000010$$

$$-7 = 100111$$

- What is the minimum number of bits needed to represent all integers in range $[-7, 7]$?
- You need 4 bits!



(a) Unsigned number



(b) Signed number

Figure 3.7 Formats for representation of integers.

2s COMPLEMENT REPRESENTATION

The SMR representation is super inefficient for arithmetic operations. Instead the 2s complement representation is very commonly used in modern computer systems.

$$\begin{array}{rccccccc} B = & b_{n-1} & b_{n-2} & \dots & b_1 & b_0 & \\ \times & -2^{n-1} & 2^{n-2} & \dots & 2^1 & 2^0 & \end{array}$$

MSB multiplied
By negative number!

$$V(B) = -2^{n-1}b_{n-1} + 2^{n-2}b_{n-2} + \dots + 2^1b_1 + 2^0b_0$$

INTERPRETING 2S COMPLEMENT

The SMR representation is super inefficient for arithmetic operations. Instead the 2s complement representation is very commonly used in modern computer systems.

$$\begin{aligned} V(B) &= -2^{n-1}b_{n-1} + \underbrace{2^{n-2}b_{n-2} + \dots + 2^1b_1 + 2^0b_0}_{M} \\ &= -2^{n-1}b_{n-1} + M \end{aligned}$$

If MSB (sign bit) is zero, $V(B) = M$

If MSB (sign bit) is 1, $V(B) = -[2^{n-1} - M]$

INTERPRETATION OF SIGNED NUMBERS

Table 3.2 Interpretation of four-bit signed integers.

$b_3b_2b_1b_0$	Sign and magnitude	1's complement	2's complement
0111	+7	+7	+7
0110	+6	+6	+6
0101	+5	+5	+5
0100	+4	+4	+4
0011	+3	+3	+3
0010	+2	+2	+2
0001	+1	+1	+1
0000	+0	+0	+0
1000	-0	-7	-8
1001	-1	-6	-7
1010	-2	-5	-6
1011	-3	-4	-5
1100	-4	-3	-4
1101	-5	-2	-3
1110	-6	-1	-2
1111	-7	-0	-1

ADDITIONS USING TWOS COMPLEMENT

Add the two numbers as if they were regular unsigned values and ignore carry-out bit from the MSB. What do you observe?

$$\begin{array}{r} (+5) \quad 0101 \\ + (+2) \quad + 0010 \\ \hline \end{array}$$


$$\begin{array}{r} (-5) \quad 1011 \\ + (+2) \quad + 0010 \\ \hline \end{array}$$


$$\begin{array}{r} (+5) \quad 0101 \\ + (-2) \quad + 1110 \\ \hline \end{array}$$


$$\begin{array}{r} (-5) \quad 1011 \\ + (-2) \quad + 1110 \\ \hline \end{array}$$


✗ Adding two numbers in 2s complement format yields the result also in 2s complement format (if we ignore carry-outs from MSBs)!

DEALING WITH OVERFLOWS

Say we want to add $-8 + -7$. These can be represented as 4-bit 2s complement numbers. BUT, the result -15 cannot be represented in 4-bits. We call this an “overflow”

How can we detect that overflow has occurred?

- Overflow can only occur when adding two positive numbers or two negative numbers.
- If the sign of the result is incorrect, then an overflow has occurred!

OVERFLOW EXAMPLE

+1

1 0 0 0 = -8

+ 1 0 0 1 = -7

× 0 0 0 1 = 1 (??!)

I added two negative numbers and got a positive result. What happened here? OVERFLOW!

OVERFLOW EXAMPLE

		+1	+1		
	0	1	1	1	= 7
+	0	0	1	0	= 2

	1	0	0	1	= -7

I added two positive numbers and got a negative result. What happened here? OVERFLOW!